Month 2 formative Questions

Learner name: Chad Teixeira and Brian Mncube
Team name: BCM
Assessor:
Project start date: 3 August
Final due date: 13 October

Question 1

Explain how the Firebase data model can prevent one user's records from being confused with or displayed as another user's records. Refer to user IDs and document ownership in your answer. ***(6 marks)***

- Every user gets a unique user ID when they register or log in with Firebase Authentication. Each record is tied to its owner in one of two ways: it is stored under that user's own path (for example users/userId/tasks/taskId), or it keeps the owner's ID inside the record (for example ownerId: userId). This is called document ownership.

- When the app loads data, it only requests records that belong to the logged-in user's ID, so each user only gets their own records back. Because every record has a clear owner, records from different users cannot be confused or displayed as someone else's.

Question 2

Explain the difference between client-side validation and authentication. Provide one example of each from the application. ***(6 marks)***

- Client-side validation checks that what the user typed is correct before it is sent. It is about the data. Example: the task form checks that the title is not empty and that the due date is a real date, and shows an error message if not.

- Authentication checks who the user is. It is about identity. Example: the user logs in with an email and password, and Firebase confirms the details before giving access to the app and their data.

Question 3

Explain when POST, GET, PUT or PATCH, and DELETE should be used in a REST workflow. Provide one application-related example for each method. ***(8 marks)***

- GET reads data without changing it. Example: load all of the logged-in learner's tasks and display them on the page.

- POST creates a new record. Example: when the learner submits the add-task form, POST saves a new task in Firebase.

- PUT or PATCH updates an existing record. PUT replaces the whole record, while PATCH changes only some fields. Example: PATCH changes a task's completed value to true when the learner ticks it off.

- DELETE removes a record. Example: DELETE removes a task from Firebase when the learner clicks the delete button.

Question 4

Explain how try, catch, finally, and throw work together when a Firebase or REST request fails. ***(8 marks)***

- The try block holds the code that might fail, such as a Firebase or fetch request. If an error happens, the code jumps straight to the catch block.
- The throw statement creates an error on purpose.
- The catch block handles the error. It shows a clear message to the user or logs the error, so the app does not crash.
- The finally block always runs, whether the request worked or failed. It is used for clean-up, such as hiding the loading spinner.

Question 5

A learner's application displays a progress total of 150% instead of 75%. Describe the steps that should be followed to identify and correct the logical error. Refer to debugging tools and the flow of data in your answer.
(8 marks)

- Open the browser Developer Tools and check the Console for errors.
- Use console.log() to print the values used in the calculation, such as the number of completed tasks and the total number of tasks.
- Follow the flow of the data: database, array, calculation and then the value shown on the screen. Find the step where the value first becomes wrong.
- Use breakpoints in the Sources tab to pause the code and inspect the variables line by line.
- Find the cause. For example, the total may be counted twice, tasks may be added to the array twice, or the result may be multiplied by 100 twice.
- Fix the formula so that it is (completed / total) * 100.
- Test again with known numbers. For example, 3 completed out of 4 tasks must show 75%.

Question 6

Explain how pull requests and peer reviews can prove individual contribution and improve the quality of a team's code. ***(6 marks)***

- A pull request shows exactly which code each learner wrote, who reviewed it and when it was merged. This gives clear proof of each member's individual contribution.

- In a peer review, a team member reads the code and leaves comments. This finds bugs and mistakes early, keeps the code neat and consistent, and helps team members learn from each other. Together they make sure the work is shared fairly and the final code is of better quality.

Question 7

A Firestore database stores learner tasks using the following structure:

```
users
└── userId
    └── tasks
        └── taskId
```

A Explain why the authenticated user's ID should be used in the document path. (2 marks)

- The authenticated user's ID comes from the login, so it is trustworthy and hard to fake. Using it in the path (users/userId/tasks) gives every user their own separate area, so tasks are never mixed up and it is easy to load and protect only that user's data.

B Explain how Firebase Security Rules can restrict access to the correct user. (3 marks)

- Security Rules compare the logged in user's ID with the userId in the path. For example: allow read, write: if request.auth != null && request.auth.uid == userId. This means only the owner can read or change their own tasks, and everyone else is blocked.

C State one risk of allowing unrestricted read and write access to the database. (1 mark)

- Anyone could read, change or delete all users' data, so private information could be stolen or the whole database could be destroyed.

D Explain why validation must still be performed even when security rules are present. (2 marks)

- Security rules only control who can access the data. They do not check whether the data itself is correct. Validation is still needed to stop empty, wrong or harmful data from being saved, and to give the user helpful feedback.

Question 8

Analyse the following JavaScript error-handling code (parts A–F follow): ***(10 marks)***

- ```
  async function saveTask(task) {
    try {
      if (!task.title) {
        throw new Error("A task title is required.");
      }
      const response = await fetch("/tasks", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(task)
      });
      const result = await response.json();
      return result;
    } catch (error) {
      console.log("Task saved successfully.");
    } finally {
      console.log("Request completed.");
    }
  }
  ```
  ```

A Explain the purpose of the async keyword. (1 mark)

- The async keyword makes a function asynchronous. It allows the function to use await and makes it always return a promise

B Explain the purpose of await. (1 mark)

- The await keyword pauses the function until a promise is finished, so the result is ready before the code continues.

C Explain why a custom error is thrown when the task has no title. (2 marks)

- The custom error stops the function from sending an invalid task. It gives a clear message that explains the problem (the title is missing), and it sends the code to the catch block so the error can be handled properly.

D Identify the incorrect message in the catch block and explain why it is misleading. (2 marks)

- The incorrect message is console.log("Task saved successfully."). It is misleading because the catch block only runs when something has gone wrong, so it tells the user the task was saved when it was not.

E Rewrite the catch block so that it reports the actual error. (2 marks)

```
catch (error) {
    console.log("Error saving task: " + error.message);
}
```

F Explain when the finally block will execute. (2 marks)

- The finally block runs every time, after the try block finishes successfully or after the catch block handles an error. It runs whether or not there was an error.

Question 9

DOM Manipulation — Study the following HTML: ***(8 marks)***

<ul id="taskList">

<button id="addTaskButton">Add Task</button>

Explain how JavaScript could be used to create a new <li> element, add task text to it, append it to the list, and later remove it. Name the relevant DOM methods or properties.

- Create the new list item: const li = document.createElement("li");
- Add the task text: li.textContent = "Buy books";.
- Find the list and append the item: document.getElementById("taskList").appendChild(li);
- The button click can be handled with document.getElementById("addTaskButton").addEventListener("click", function).
- Later, remove the item with li.remove(), or with taskList.removeChild(li).

Question 10

Object-Oriented JavaScript — Study the following class: ***(8 marks)***

```
class Task {
  constructor(title, dueDate) {
    this.title = title;
    this.dueDate = dueDate;
    this.completed = false;
  }
  markComplete() {
    this.completed = true;
  }
}
```

A Explain the purpose of the Task class. (2 marks)

- The Task class is a blueprint used to create task objects. Every task made from it has the same properties and methods.

B Explain what the constructor does. (2 marks)

- The constructor is a special method that runs automatically when a new Task object is created. It sets the starting values of the object's properties using the values that are passed in.

C State the three properties created for every task object. (2 marks)

- title, dueDate and completed.

D Explain what happens when markComplete() is called. (1 mark)

- It changes the task's completed property to true, which means the task is marked as done.

E Write one statement that creates a new Task object. (1 mark)

- const task1 = new Task("Study JavaScript", "2026-10-30");

Question 11

Explain how JavaScript timers such as setTimeout() and setInterval() differ. Describe one suitable use for each in a web application, and state one consideration when adding animation or multimedia. ***(6 marks)***

- setTimeout() runs code once, after a delay. Use: hide a "Task saved" message after 3 seconds.
- setInterval() runs code again and again at a set interval until it is stopped with clearInterval(). Use: update a countdown timer or clock every second.
- Consideration: keep animations and multimedia light and under the user's control (for example pause or mute buttons), so they do not slow the page or annoy the user. Clear timers when they are no longer needed.

Question 12

A learner submits a form that creates a new record in Firebase. The application must then retrieve the record and display it on the page. Describe the complete asynchronous workflow from the moment the user selects the submit button until the new record appears in the interface. Refer to validation, the database request, the response, DOM updates, loading feedback, and error handling. ***(8 marks)***

- The user clicks the Submit button.
- JavaScript stops the page from reloading using event.preventDefault().
- Validation: the input is checked. If it is invalid, an error message is shown and the process stops.
- Loading feedback: a "Saving..." message or spinner is shown.
- Database request: the data is sent to Firebase (POST or set) using async/await.
- Response: the app waits for Firebase to confirm that the record was saved.
- The new record is retrieved from the database (GET).
- DOM update: a new element is created with the record's data and added to the page, and the form is cleared.
- Error handling: if anything fails, the catch block shows a clear error message to the user.
- The finally block hides the loading spinner, whether it worked or not.

Academic Integrity and Authenticity Declaration

- I declare that:
- the evidence submitted for assessment accurately represents my own contribution;
- I have acknowledged tutorials, libraries, code examples, artificial-intelligence assistance and other external sources;
- I understand the JavaScript, Firebase and GitHub work that I am presenting;
- I can explain, test and modify the code attributed to me;
- I have not stored passwords, private credentials or confidential data in the repository;
- I understand that if I cannot demonstrate understanding OR provide enough evidence, my project could not be accepted and therefore I will not pass the FISA (Final Integrated Supervised Assessment).

Learner name: Chad Teixeira and Brian Mncube

Learner signature: CT BM

Date: 13 October 2026

Assessor name: _____

Assessor signature: _____

Date: _____